Computer Vision - Project 1: Ingenious Score Calculator

Dominic Armand Stamatoiu

May 2026

1 Introduction

This report details the implementation of an automated computer vision pipeline designed to track and calculate scores for the board game Ingenious. The system is tasked with processing a sequence of 20 top-down photographs of the game board per match. For each turn, the pipeline must accurately extract the positions of the newly placed tile, determine the color of its two segments, calculate the points earned according to the game's rules, and match the updated player score to a corresponding scorecard image from a database of 200 images.

2 Board Alignment & Grid Mapping

Because the images are captured under varying perspectives, scales, and rotations, the first step is to warp each current image to match a static, empty reference board (`board_1.jpg`).

Image Alignment (SIFT & Homography):
To ensure reliable feature extraction, we first apply CLAHE (Contrast Limited Adaptive Histogram Equalization) to the grayscale image. This normalizes local lighting and removes harsh shadows. We then use the SIFT (Scale-Invariant Feature Transform) algorithm to detect keypoints and compute descriptors. Using a FLANN-based matcher (KD-Tree algorithm), we find the best matches between the current image and the reference board, applying Lowe's Ratio Test (with a threshold of 0.7) to filter out ambiguous matches. Finally, we compute a Homography matrix using RANSAC with a 5.0 pixel tolerance and warp the perspective of the current image so it perfectly aligns with our reference frame.

Grid Calibration:
With a perfectly aligned image, we project a predefined hexagonal grid onto the board. The board center is hardcoded to pixel coordinates $(2055, 1460)$, with horizontal and vertical spacing defined as DX = 144 and DY = 124. Using a mathematical array representing the row lengths ranging from 4 to 10 playable spaces, we apply offsets to calculate the exact pixel coordinates of all 85 playable hexes.

3 Tile Position Detection

To detect the newly placed tile, we employ a background subtraction technique, comparing the current aligned image to the previous turn's image.

Both images are converted to grayscale and smoothed using a 5×5 Gaussian Blur to eliminate minor noise and camera artifacts. For every 85 valid grid positions, we create a circular mask with

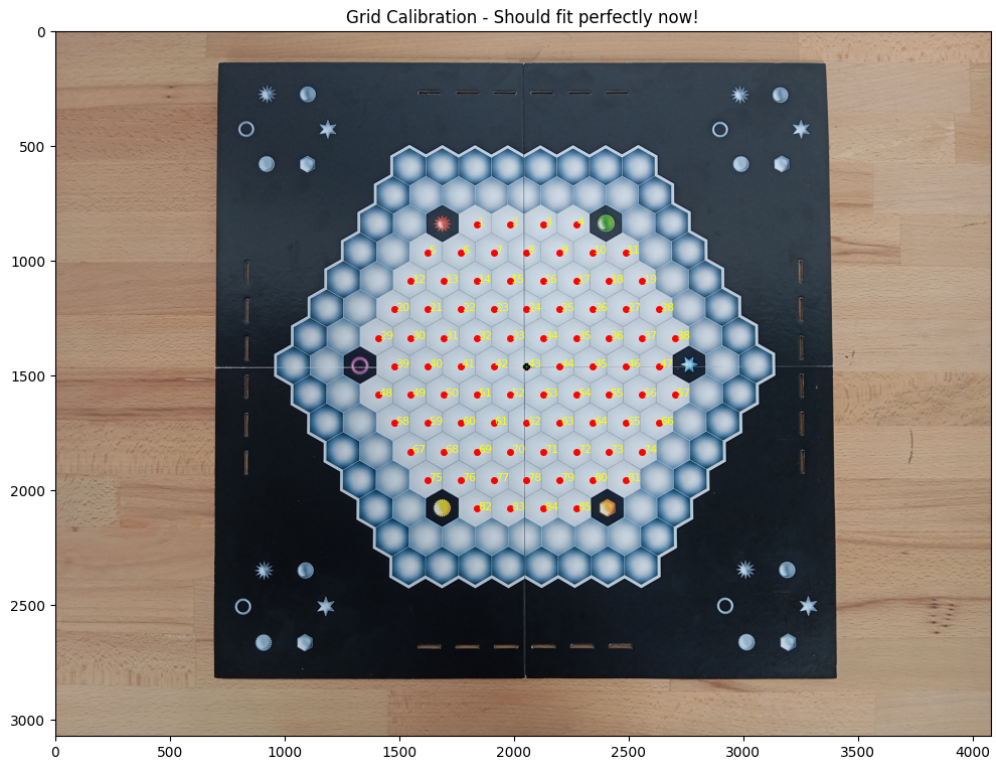

Figure 1: Grid Calibration overlay on the aligned board

a radius of 60 pixels and calculate the mean brightness inside that area for both the previous and current turns.

Since the empty board is bright white and the tiles are dark plastic, a newly placed tile causes a significant drop in brightness. We calculate the difference (mean_prev − mean_curr). To prevent the algorithm from double-counting tiles placed on previous turns, we implement a guardrail: if mean_prev < 60, the space is already occupied, and the drop is manually set to -999. The system simply sorts the cells by the largest brightness drop and selects the top two indices.

4 Color Classification

Once the two new positions are identified, we must determine their colors. Identifying color under variable room lighting is challenging, so we convert the warped image into the HSV (Hue, Saturation, Value) color space. We isolate the Value brightness channel, to apply CLAHE to flatten out glares and shadows, and merge it back.

For each of the two detected positions, we create a circular mask with a radius of 20 pixels. To account for the camera's slight physical angle capturing the raised plastic symbols, we manually shift the sampling center up by 30 pixels (y_pos − 30).

Instead of averaging the color, which could be skewed by the black background of the tile, we implement a pixel voting system. We iterate through every pixel inside the mask and categorize its Hue into one of six predefined ranges: Red, Green, Orange, Blue, Yellow or Purple. Because Red sits at the 0° mark on the HSV cylinder, its range wraps around the spectrum (values ≤ 8 or ≥ 165). The color category that receives the highest number of pixel votes is selected as the final prediction.

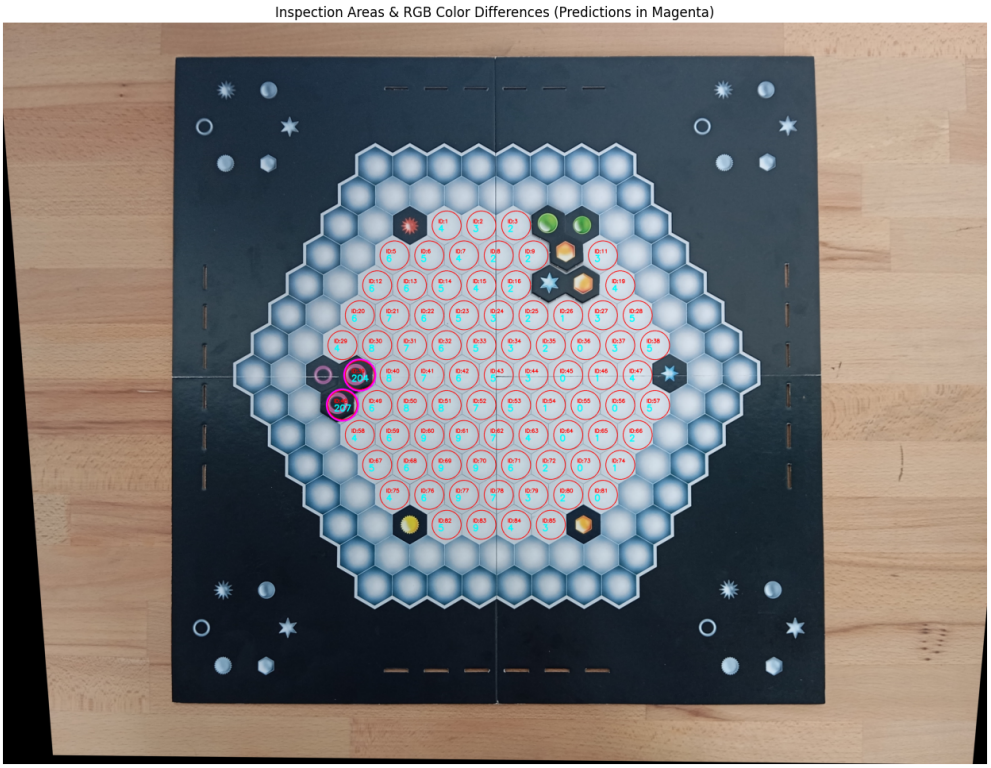

Figure 2: Brightness drop inspection areas

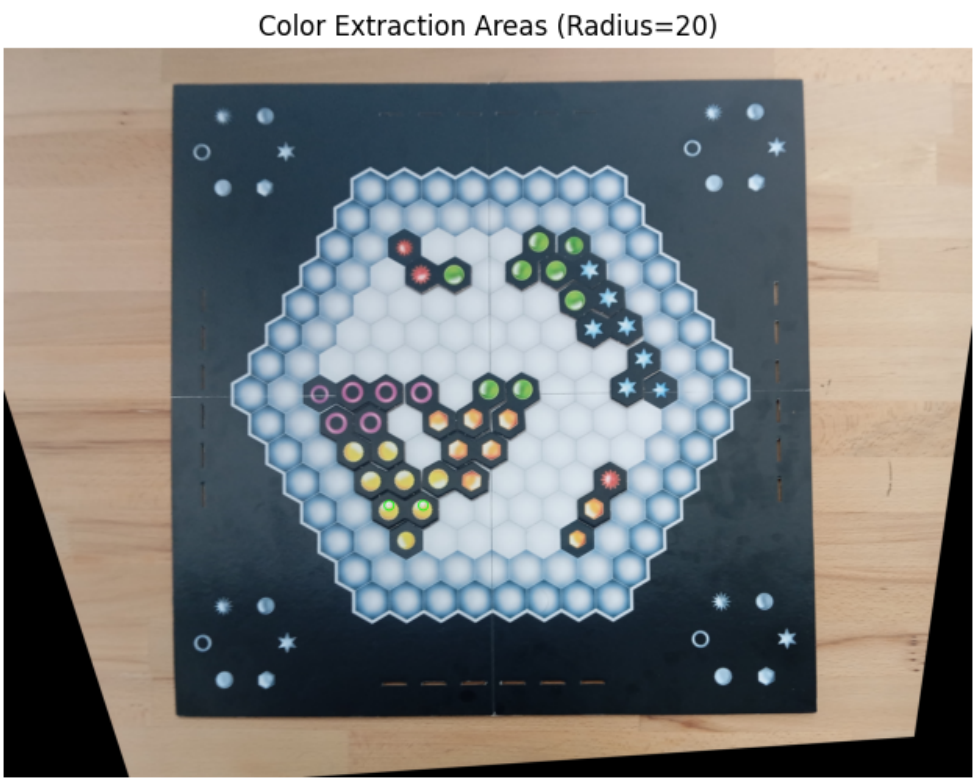

Figure 3: Color extraction areas and RGB color differences

5 Logical Mapping & Score Calculation

To calculate the score, the physical pixel map is translated into a logical mathematical grid. By staggering the x coordinates of alternating rows by 0.5 and 0.0, we create a Cartesian map where hexagonal neighbors can be found using universal offsets (top-right is $x + 0.5$, $y - 1$). We also manually map the 6 pre-printed border symbols into this abstract coordinate system.

For each half of the newly placed tile, the scoring function queries our compass function to find adjacent occupied cells. It then loops through all 6 hexagonal directions. As long as it steps onto a cell that exists in our `current_board_state` memory and matches the starting segment's color, it increments the score by 1 and steps forward again in the exact same direction (dx, dy). The loop breaks if it encounters a different color, an empty space, or the edge of the board. The scores for both segments are tallied independently.

6 Scoreboard Extraction and Mapping

The final step is to locate the correct table scorecard image from a database of 200 images.

We process all 200 images by applying a Gaussian blur followed by Canny Edge Detection to ignore shadows and strictly find the sharp cardboard edges. We use dilation to ensure the board outline is a closed loop and extract the bounding box (`bx, by, bw, bh`) of the largest contour.

Based on the total height of the bounding box, we define the 18-point line as roughly 7% down from the top edge, and the 0-point line as 83% down. We convert the image to HSV and use `cv2.inRange` to create masks for each of the 6 colored scoring cubes. After finding the vertical center pixel (`center_y`) of each cube, we use linear interpolation to determine the score:

$$\text{raw_score} = \left(\frac{\text{y_min_score} - \text{center_y}}{\text{span}} \right) \times 18 \tag{1}$$

y_min_score being the 0-point line and span, the distance between the 0-point and 18-point lines.

The result is bounded between 0 and 18. This creates a dictionary database of all 200 table scores. During the main game loop, our system adds the newly calculated move points to the active player's cumulative total, and then simply searches the database for the image filename that perfectly matches this new score state.

0198.jpg | Calculated Score: {'R': 4, 'G': 4, 'O': 4, 'B': 3, 'Y': 0, 'P': 15}

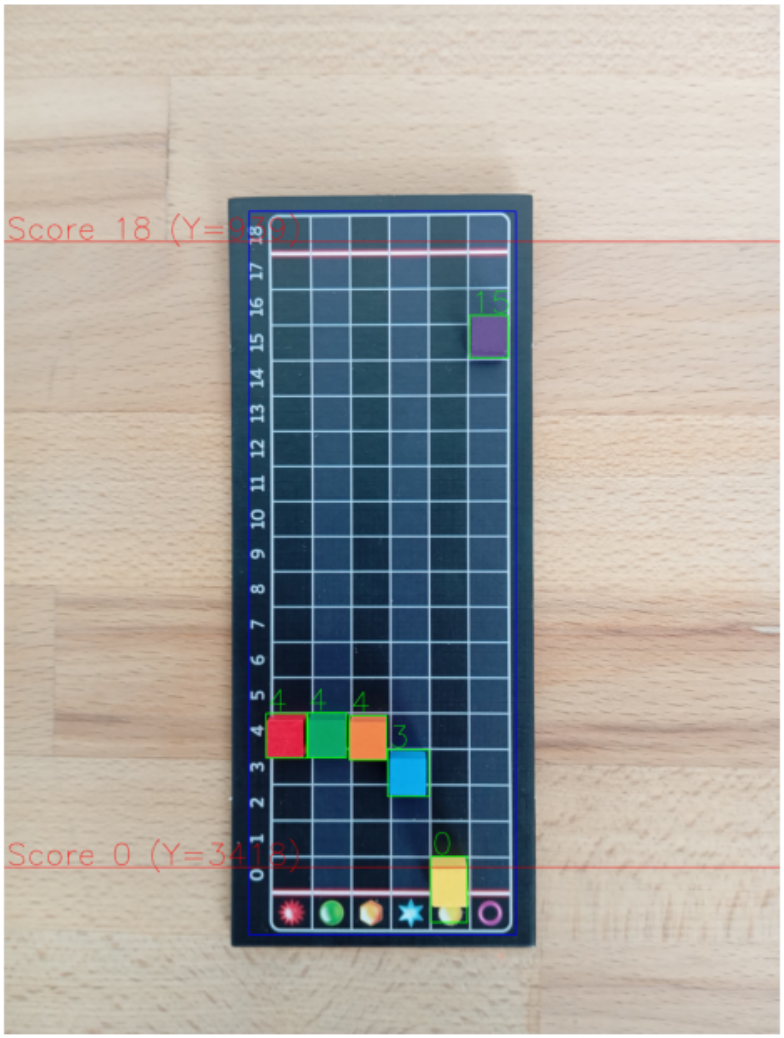

Figure 4: Table score boundary detection and calculated linear interpolation